

Problem Set 4

BM2043 - Algorithms and Data Structure Lab

October 1, 2024

Binary Tree and Binary Search Tree Implementation

1. Implementation of a Binary Tree with Traversals

Objective: Implement a binary tree data structure from scratch without using the Standard Template Library (STL). The binary tree should support insertion of nodes and provide depth-first (DFS) and breadth-first (BFS) traversal methods.

Instructions: Define a `BinaryTree` class with the following methods:

- **insert(int value):** Inserts a new integer value into the binary tree.

Depth-First Search (DFS) Traversals:

- **preorderTraversal():** Performs a pre-order traversal (root, left, right) of the binary tree and returns the values in an array or vector.
- **inorderTraversal():** Performs an in-order traversal (left, root, right) of the binary tree and returns the values in an array or vector.
- **postorderTraversal():** Performs a post-order traversal (left, right, root) of the binary tree and returns the values in an array or vector.

Breadth-First Search (BFS) Traversals:

- **levelOrderTraversal():** Performs a level-order traversal (BFS) of the binary tree and returns the values in an array or vector.
- **zigzagTraversal():** Performs a zigzag traversal (alternating levels) of the binary tree and returns the values in an array or vector.

The binary tree should be implemented using a `TreeNode` structure, which will represent each node in the tree.

Class Definition:

```
class TreeNode {
public:
    int value;
    TreeNode* left;
    TreeNode* right;

    TreeNode(int val) : value(val), left(nullptr), right(nullptr) { }
};
```

Binary Tree Class:

```
class BinaryTree {
private:
    TreeNode* root;

public:
    BinaryTree(); // Constructor to initialize an empty tree
    void insert(int value); // Inserts a new value into the binary tree

    // Depth-First Search (DFS) Methods
    vector<int> preorderTraversal(); // Returns an array of values from pre-order traversal
    vector<int> inorderTraversal(); // Returns an array of values from in-order traversal
    vector<int> postorderTraversal(); // Returns an array of values from post-order traversal

    // Breadth-First Search (BFS) Methods
    vector<int> levelOrderTraversal(); // Returns an array of values from level-order traversal
    vector<int> zigzagTraversal(); // Returns an array of values from zigzag traversal

private:
    void preorderHelper(TreeNode* node, vector<int>& result); // Helper for pre-order traversal
    void inorderHelper(TreeNode* node, vector<int>& result); // Helper for in-order traversal
    void postorderHelper(TreeNode* node, vector<int>& result); // Helper for post-order traversal
    void levelOrderHelper(TreeNode* node, vector<vector<int>>& result); // level-order traversal
    void zigzagHelper(TreeNode* node, vector<vector<int>>& result); // Helper for zigzag traversal
};
```

2. Implementation of a Binary Search Tree with Traversals

Objective: Implement a binary search tree (BST) data structure from scratch without using the Standard Template Library (STL). The BST should support insertion, deletion, and various traversals.

Instructions: Define a `BinarySearchTree` class with the following methods:

- **insert(int value):** Inserts a new integer value into the binary search tree.
- **delete(int value):** Deletes a specified integer value from the binary search tree.

Depth-First Search (DFS) Traversals:

- **preorderTraversal():** Performs a pre-order traversal (root, left, right) of the binary search tree and returns the values in an array or vector.
- **inorderTraversal():** Performs an in-order traversal (left, root, right) of the binary search tree and returns the values in an array or vector.
- **postorderTraversal():** Performs a post-order traversal (left, right, root) of the binary search tree and returns the values in an array or vector.

Breadth-First Search (BFS) Traversals:

- **levelOrderTraversal():** Performs a level-order traversal (BFS) of the binary search tree and returns the values in an array or vector.

- **zigzagTraversal():** Performs a zigzag traversal (alternating levels) of the binary search tree and returns the values in an array or vector.

The binary search tree should be implemented using a `TreeNode` structure, which will represent each node in the tree.

Class Definition:

```
class TreeNode {
public:
    int value;
    TreeNode* left;
    TreeNode* right;

    TreeNode(int val) : value(val), left(nullptr), right(nullptr) { }
};
```

Binary Search Tree Class:

```
class BinarySearchTree {
private:
    TreeNode* root;

public:
    BinarySearchTree();                // Constructor to initialize an empty BST
    void insert(int value);            // Inserts a new value into the binary search tree
    void delete(int value);            // Deletes a specified value from the binary search tree

    // Depth-First Search (DFS) Methods
    vector<int> preorderTraversal();    // Returns an array of values from pre-order traversal
    vector<int> inorderTraversal();     // Returns an array of values from in-order traversal
    vector<int> postorderTraversal();   // Returns an array of values from post-order traversal

    // Breadth-First Search (BFS) Methods
    vector<int> levelOrderTraversal();  // Returns an array of values from level-order traversal
    vector<int> zigzagTraversal();      // Returns an array of values from zigzag traversal

private:
    void insertHelper(TreeNode*& node, int value); // Helper for insertion
    void deleteHelper(TreeNode*& node, int value); // Helper for deletion
    void preorderHelper(TreeNode* node, vector<int>& result); // Helper for pre-order traversal
    void inorderHelper(TreeNode* node, vector<int>& result);  // Helper for in-order traversal
    void postorderHelper(TreeNode* node, vector<int>& result); // Helper for post-order traversal
    void levelOrderHelper(TreeNode* node, vector<vector<int>>& result); //level-order traversal
    void zigzagHelper(TreeNode* node, vector<vector<int>>& result); // Helper for zigzag traversal
};
```

Problems on Binary Trees and Binary Search Trees

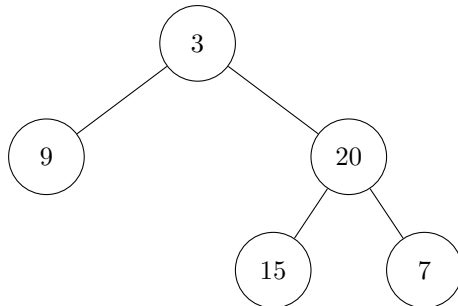
Problem 1: Maximum Depth / Height of Binary Tree

Objective: Given the root of a binary tree, return its maximum depth.

The maximum depth of a binary tree is defined as the number of nodes along the longest path from the root node down to the farthest leaf node.

Examples:

Example 1:



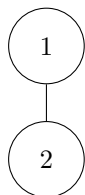
• **Input:**

`root = [3,9,20,null,null,15,7]`

• **Output:**

3

Example 2:



• **Input:**

`root = [1,2]`

• **Output:**

2

Constraints:

- The number of nodes in the tree is in the range $[0, 10^4]$.
- $-100 \leq \text{Node.val} \leq 100$.

Class Definition:

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
 * };
 */
class Solution {
public:
    int maxDepth(TreeNode* root) {
        // Implement the logic to calculate the maximum depth of the binary tree
    }
};
```

Problem 2: Diameter of the binary tree

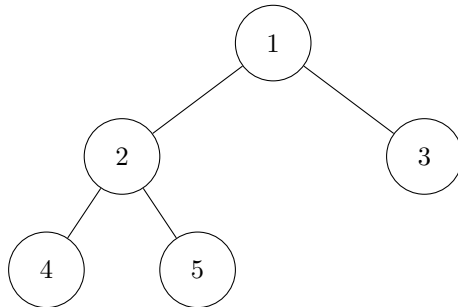
Objective: Given the root of a binary tree, return the length of the diameter of the tree.

The diameter of a binary tree is defined as the length of the longest path between any two nodes in the tree. This path may or may not pass through the root.

The length of a path between two nodes is represented by the number of edges between them.

Examples:

Example 1:



- **Input:**

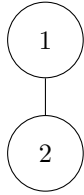
root = [1,2,3,4,5]

- **Output:**

3

- **Explanation:**

3 is the length of the path [4,2,1,3] or [5,2,1,3].

Example 2:• **Input:**

root = [1,2]

• **Output:**

1

Constraints:

- The number of nodes in the tree is in the range $[1, 10^4]$.
- $-100 \leq \text{Node.val} \leq 100$.

Class Definition:

```

/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
 * };
 */
class Solution {
public:
    int diameterOfBinaryTree(TreeNode* root) {
        // Implement the logic to calculate the diameter of the binary tree
    }
};

```

Problem 3: Maximum Path Sum in Binary Tree

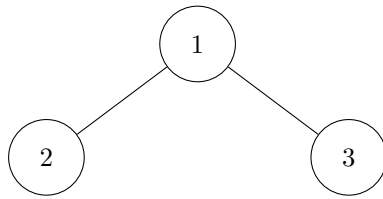
Objective: A path in a binary tree is a sequence of nodes where each pair of adjacent nodes in the sequence has an edge connecting them. A node can only appear in the sequence at most once. Note that the path does not need to pass through the root.

The path sum of a path is the sum of the node's values in the path.

Given the root of a binary tree, return the maximum path sum of any non-empty path.

Examples:

Example 1:



• **Input:**

`root = [1,2,3]`

• **Output:**

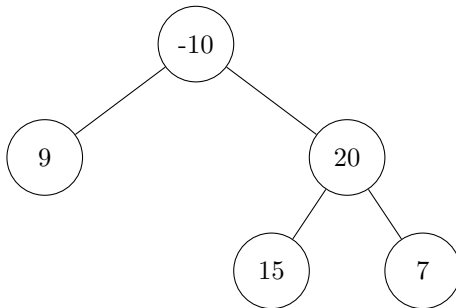
6

• **Explanation:**

The optimal path is 2 -> 1 -> 3 with a path sum of $2 + 1 + 3 = 6$.

Example 2:

• **Input:**



`root = [-10,9,20,null,null,15,7]`

• **Output:**

42

• **Explanation:**

The optimal path is 15 -> 20 -> 7 with a path sum of $15 + 20 + 7 = 42$.

Constraints:

- The number of nodes in the tree is in the range $[1, 3 \times 10^4]$.
- $-1000 \leq \text{Node.val} \leq 1000$.

Class Definition:

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
 * };
 */
class Solution {
public:
    int maxPathSum(TreeNode* root) {
        // Implement the logic to calculate the maximum path sum
    }
};
```

Problem 4: Lowest Common Ancestor of a Binary Tree

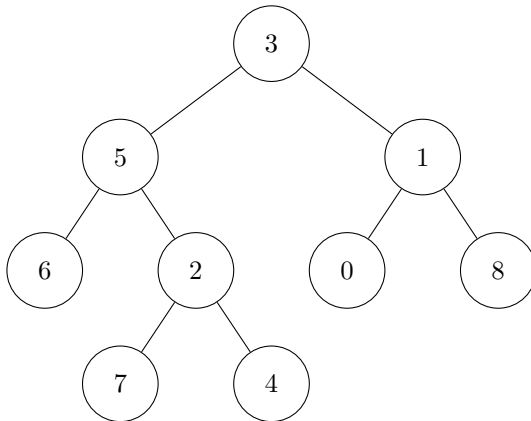
Objective: Given a binary tree, find the lowest common ancestor (LCA) of two given nodes in the tree.

According to the definition of LCA: “The lowest common ancestor is the lowest node in the tree that has both nodes p and q as descendants (where we allow a node to be a descendant of itself).”

Examples:

Example 1:

• Input:



root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1

• Output:

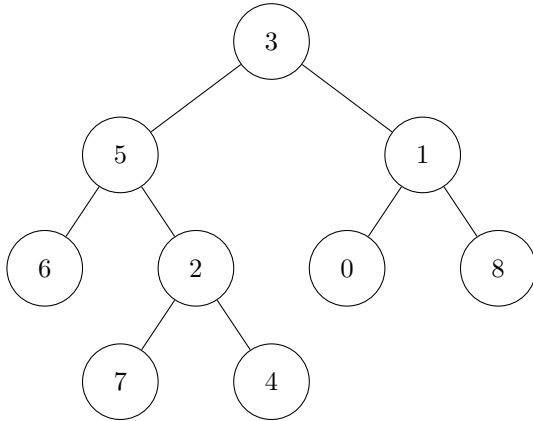
3

- **Explanation:**

The LCA of nodes 5 and 1 is 3.

Example 2:

- **Input:**



root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 4

- **Output:**

5

- **Explanation:**

The LCA of nodes 5 and 4 is 5, since a node can be a descendant of itself.

Example 3:

- **Input:**

root = [1,2], p = 1, q = 2

- **Output:**

1

Constraints:

- The number of nodes in the tree is in the range $[2, 10^5]$.
- $-10^9 \leq \text{Node.val} \leq 10^9$.
- All Node.val are unique.
- $p \neq q$.
- Both p and q will exist in the tree.

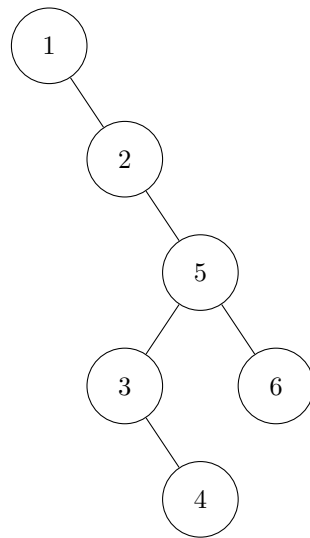
Class Definition:

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode(int x) : val(x), left(NULL), right(NULL) {}
 * };
 */
class Solution {
public:
    TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
        // Implement the logic to find the lowest common ancestor
    }
};
```

Problem 5: Top View of a Binary Tree

Objective: Given a pointer to the root of a binary tree, print the top view of the binary tree. The top view of the tree is defined as the nodes that are visible when the tree is viewed from the top.

Examples:



Example:

- **Output:**

1 2 5 6

- **Explanation:**

The top view of the tree includes nodes 1, 2, 5, and 6.

Constraints:

- The number of nodes in the binary tree is in the range $[1, 10^4]$.
- The value of each node in the tree is within the range $[-1000, 1000]$.

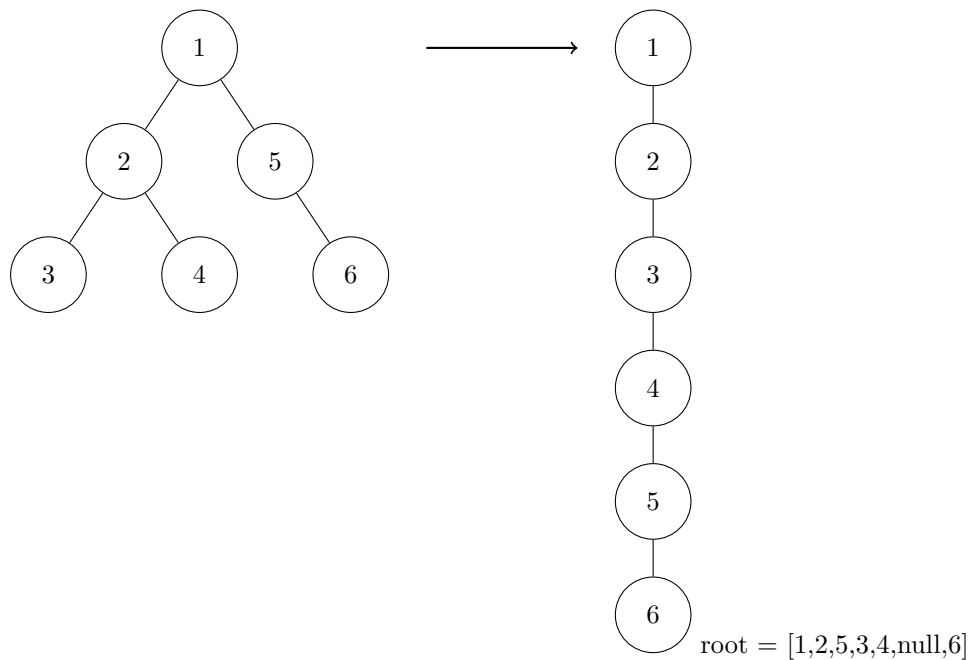
Function Definition:

```
void topView(node * root) {  
    // Complete the function to print the top view of the tree  
}
```

Problem 6: Flatten Binary Tree to Linked List

Objective: Given the root of a binary tree, flatten the tree into a "linked list". The "linked list" should use the same `TreeNode` class where:

- The right child pointer points to the next node in the list.
- The left child pointer is always `null`.
- The "linked list" should be in the same order as a pre-order traversal of the binary tree.

Examples:**Example 1:**

- **Input:**
- **Output:** `[1,null,2,null,3,null,4,null,5,null,6]`

Example 2:

- **Input:** `root = []`
- **Output:** `[]`

Example 3:

- **Input:** root = [0]
- **Output:** [0]

Constraints:

- The number of nodes in the tree is in the range $[0, 2000]$.
- The value of each node is within the range $[-100, 100]$.

Follow-up: Can you flatten the tree **in-place** (with $O(1)$ extra space)?

Class Definition:

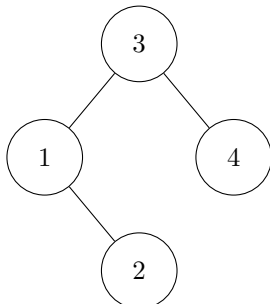
```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
 * };
 */
class Solution {
public:
    void flatten(TreeNode* root) {
        // Implement the logic to flatten the binary tree into a linked list
    }
};
```

Problem 7: Kth Smallest Element in a Binary Search Tree (BST)

Objective: Given the root of a binary search tree (BST) and an integer k , return the k -th smallest value (1-indexed) of all the values of the nodes in the tree.

Examples:**Example 1:**

- **Input:**

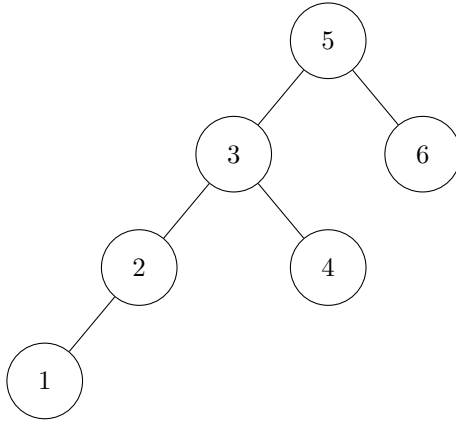


root = [3,1,4,null,2], k = 1

- **Output:** 1

Example 2:

- **Input:**



root = [5,3,6,2,4,null,null,1], k = 3

- **Output:** 3

Constraints:

- The number of nodes in the tree is n .
- $1 \leq k \leq n \leq 10^4$.
- $0 \leq \text{Node.val} \leq 10^4$.

Follow-up: If the BST is modified often (i.e., we can perform insert and delete operations) and you need to find the k -th smallest frequently, how would you optimize the solution?

Class Definition:

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
 * };
 */
class Solution {
public:
    int kthSmallest(TreeNode* root, int k) {
        // Implement logic to find kth smallest element
    }
};
```

Problem 8: Lowest Common Ancestor in a Binary Search Tree (BST)

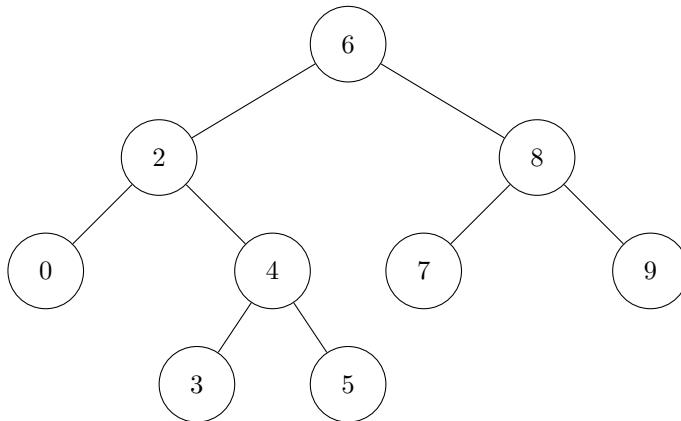
Objective: Given a binary search tree (BST), find the lowest common ancestor (LCA) node of two given nodes in the BST.

According to the definition of LCA: "The lowest common ancestor is defined between two nodes p and q as the lowest node in T that has both p and q as descendants (where we allow a node to be a descendant of itself)."

Examples:

Example 1:

• **Input:**

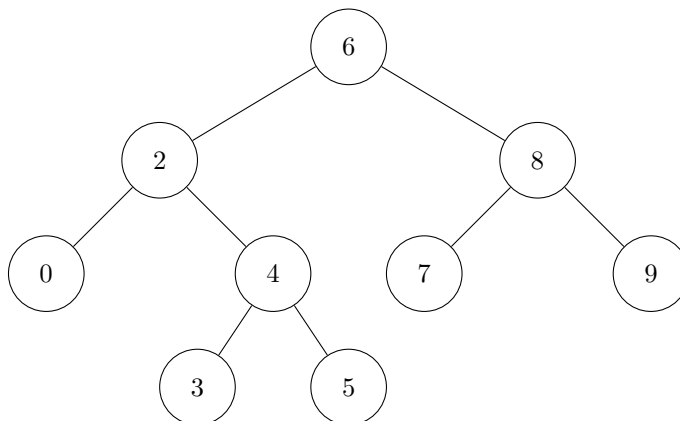


root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 8

• **Output:** 6

• **Explanation:** The LCA of nodes 2 and 8 is 6.

Example 2:



• **Input:**

p = 2, q = 4

root = [6,2,8,0,4,7,9,null,null,3,5],

• **Output:** 2

• **Explanation:** The LCA of nodes 2 and 4 is 2, since a node can be a descendant of itself according to the LCA definition.

Example 3:

- **Input:** root = [2,1], p = 2, q = 1
- **Output:** 2

Constraints:

- The number of nodes in the tree is in the range $[2, 10^5]$.
- $-10^9 \leq \text{Node.val} \leq 10^9$.
- All Node.val are unique.
- $p \neq q$.
- p and q will exist in the BST.

Class Definition:

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode(int x) : val(x), left(NULL), right(NULL) {}
 * };
 */
class Solution {
public:
    TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
        // Implement logic to find the lowest common ancestor of nodes p and q
    }
};
```

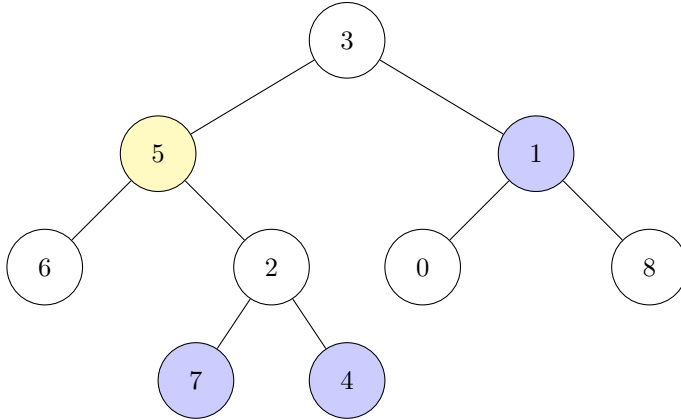
Problem 9: All Nodes Distance K in Binary Tree

Objective: Given the root of a binary tree, the value of a target node *target*, and an integer *k*, return an array of the values of all nodes that have a distance *k* from the target node.

You can return the answer in any order.

Examples:**Example 1:**

- **Input:**



root = [3,5,1,6,2,0,8,null,null,7,4], target = 5, k = 2

- **Output:** [7, 4, 1]
- **Explanation:** The nodes that are a distance 2 from the target node (with value 5) have values 7, 4, and 1.

Example 2:

- **Input:** root = [1], target = 1, k = 3
- **Output:** []

Constraints:

- The number of nodes in the tree is in the range [1, 500].
- $0 \leq \text{Node.val} \leq 500$.
- All the values Node.val are unique.
- *target* is the value of one of the nodes in the tree.
- $0 \leq k \leq 1000$.

Class Definition:

```

/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode(int x) : val(x), left(NULL), right(NULL) {}
 * };
 */
class Solution {
public:
    vector<int> distanceK(TreeNode* root, TreeNode* target, int k) {
        // Implement logic to find all nodes at distance k from the target node
    }
};

```